PyTorch From Zero

Deep Learning course notes Sources: listed under References

What this file is really about:

$$\text{tensors} \rightarrow \text{gradients} \rightarrow \text{modules} \rightarrow \text{data} \rightarrow \text{a training loop you can trust} \rightarrow \text{speed and debugg}$$

How to use this file. Part A explains each piece of the library and why it exists. Part B is complete, runnable code: copy each program into one file and run it. Part C is practice with answers.

Step-up path

You need. Python and the idea of a gradient.

1. Move tensors and read their shapes.
2. Compute gradients with autograd.
3. Write a module and a training loop.
4. Debug by overfitting one batch.
5. Scale up with mixed precision and checkpoints.

You can stop when you can write and debug a training pipeline without a template.

Contents

Part A Maths From the Ground Up

A1 A tensor is an array that remembers

A tensor is an n-dimensional array of numbers. It adds three things to a NumPy array: it can live on a GPU, it can record how it was computed, and its operations can be differentiated.

Every tensor has a **shape**, a **data type** and a **device**. Most bugs in deep learning code are shape bugs, so read shapes as you would read units in physics.

A batch of B colour images of side H has shape $B \times 3 \times H \times H$ in PyTorch. A batch of B sequences of T tokens, each embedded in d dimensions, has shape $B \times T \times d$.

A2 Broadcasting

When two tensors have different shapes, PyTorch aligns them from the **right**. Two dimensions are compatible if they are equal or if one of them is 1, and a dimension of size 1 is stretched to match.

> **Worked example.** Shapes $(32, 3)$ and (3) give $(32, 3)$. Shapes $(32, 1, 3)$ and $(1, 10, 3)$ give $(32, 10, 3)$, which is how all pairwise differences between two sets of points are computed in one line.

> **Careful.** Shapes (32) and $(32, 1)$ broadcast to $(32, 32)$. A loss computed as `(pred - y) ** 2` with those two shapes silently compares every prediction with every target. This is one of the most common silent bugs.

A3 Gradients and the computation graph

Setting `requires_grad=True` asks PyTorch to record every operation on a tensor. `loss.backward()` walks that record backwards, applying the chain rule, and adds the result to each leaf's `.grad`.

Three rules follow from "adds":

- Gradients accumulate, so clear them before each backward pass.
- The graph is freed after `backward()`, unless you ask to retain it.
- Tensors you only want to read should be detached, or computed inside `torch.no_grad()`.

A4 Modules and parameters

An `nn.Module` is a container of parameters and sub-modules. Assigning an `nn.Parameter` or another module as an attribute registers it automatically, so `model.parameters()` finds everything.

`model.train()` and `model.eval()` switch the behaviour of layers such as dropout and batch normalisation. Forgetting `model.eval()` is another common bug: dropout stays on during evaluation, and scores look noisy.

A5 Losses and logits

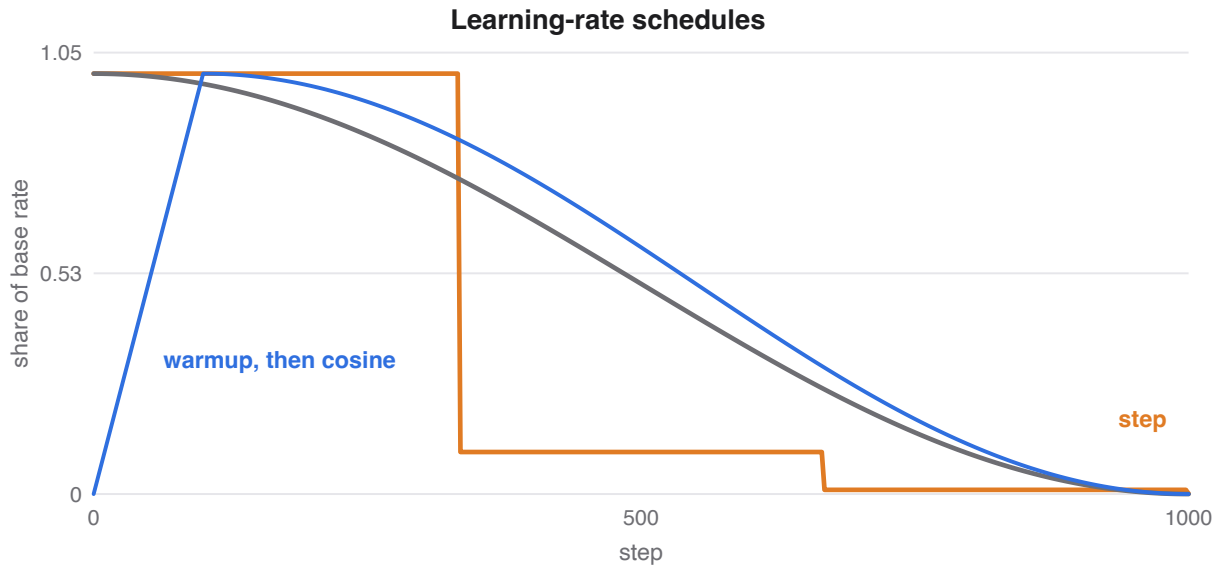
`nn.CrossEntropyLoss` expects raw scores, called **logits**, and applies log-softmax itself, because computing $\log(\mathrm{softmax}(z))$ in one step is numerically stable. Applying softmax yourself first gives a double softmax. Training still runs, but works poorly.

Shapes matter here too. For a batch of B examples and K classes the logits are $B \times K$ and the targets are integer class indices of shape B. For sequences, flatten both first.

A6 Optimisers

An optimiser holds references to parameters. `opt.step()` applies an update using each `.grad`. `AdamW` applies weight decay directly to the weights, which is correct, whereas plain `Adam` with `weight_decay` adds it to the gradient, where it interacts with the adaptive scaling.

Weight decay is usually not applied to biases and normalisation parameters. They do not control the sharpness of the function in the same way.



A7 Data

A `Dataset` answers two questions: how many items exist and what item i is. A `DataLoader` batches, shuffles, and optionally loads in parallel.

- Shuffle training data. Do not shuffle validation data.
- `num_workers` loads in background processes.
- `pin_memory=True` speeds up transfer to a GPU.
- `drop_last=True` avoids a final small batch that would break batch normalisation.

A8 The training loop

Every training loop has the same five lines at its heart:

1. Clear old gradients.
2. Compute predictions.
3. Compute the loss.
4. Compute new gradients.
5. Take a step.

Around them: move data to the device, track metrics, evaluate on held-out data in `no_grad`, and save checkpoints.

A9 Debugging a network

Work through these in order. Each step rules out a whole class of problem.

1. **Check the first loss.** For K balanced classes, an untrained model should give about $\ln K$. If it does not, the output layer or loss is wrong.
2. **Overfit one batch.** A healthy model drives the loss to nearly zero on a single batch. If it cannot, there is a bug, not a capacity problem.
3. **Check gradients flow.** Print the gradient norm per layer. Zeros mean something is detached. Enormous values mean explosion.
4. **Check the data.** Look at a batch, with its labels, after all transforms.
5. **Only then tune.** Start with the learning rate.

A10 Speed

- **GPU transfer.** Keep the model and data on the same device.
- **Mixed precision.** Using 16-bit floats where safe roughly doubles speed and halves memory. A `GradScaler` protects small gradients from underflow.
- **Gradient accumulation.** To simulate a batch k times larger, divide the loss by k and step every k batches.
- **Clipping.** Rescale gradients whose total norm exceeds a limit: $g \leftarrow g \cdot \min(1,\, c/\|g\|)$.
- **Distributed training.** `DistributedDataParallel` runs one process per GPU, and averages gradients between them.

A11 Saving and resuming

Save the model's `state_dict`, the optimiser's state, the epoch, and the random generators' state. Without the optimiser's state, resuming with Adam restarts its running averages.

Part B In Code

B1 Tensors and broadcasting

```python
import torch

x = torch.tensor([[1., 2.], [3., 4.]])
print(x.shape, x.dtype, x.device)       # torch.Size([2, 2]) float32 cpu

a = torch.randn(32, 3)                  # 32 points in 3-D
b = torch.randn(10, 3)                  # 10 centres
d = ((a[:, None, :] - b[None, :, :]) ** 2).sum(-1)
print(d.shape)                          # torch.Size([32, 10])

pred = torch.randn(32)
y = torch.randn(32, 1)
print((pred - y).shape)                 # torch.Size([32, 32])  the silent bug
print((pred - y.squeeze(1)).shape)      # torch.Size([32])      correct
```

B2 Gradients by hand

```
import torch

x = torch.tensor(2.0, requires_grad=True)
y = x ** 3 + 2 * x
y.backward()
print(x.grad)                          # 3 * 2**2 + 2 = 14

y2 = x ** 2
y2.backward()
print(x.grad)                          # 18: 14 + 4, gradients accumulate

x.grad = None
with torch.no_grad():
    z = x * 5
print(z.requires_grad)                 # False
```

B3 A complete classifier, from nothing

This program trains a small network on synthetic data, and evaluates it properly.

```
import torch
from torch import nn
from torch.utils.data import DataLoader, TensorDataset, random_split

torch.manual_seed(0)
device = 'cuda' if torch.cuda.is_available() else 'cpu'

# 1. Data: three Gaussian blobs in 2-D.
n = 600
centres = torch.tensor([[-2., 0.], [2., 0.], [0., 3.]])
y = torch.randint(0, 3, (n,))
X = centres[y] + 0.8 * torch.randn(n, 2)
data = TensorDataset(X, y)
train_set, val_set = random_split(data, [480, 120])
train = DataLoader(train_set, batch_size=32, shuffle=True)
val = DataLoader(val_set, batch_size=256)

# 2. Model. The last layer outputs logits.
model = nn.Sequential(nn.Linear(2, 64), nn.ReLU(),
                      nn.Linear(64, 64), nn.ReLU(),
                      nn.Linear(64, 3)).to(device)

# 3. Loss and optimiser.
loss_fn = nn.CrossEntropyLoss()
opt = torch.optim.AdamW(model.parameters(), lr=1e-3, weight_decay=1e-4)

@torch.no_grad()
def accuracy(loader):
    model.eval()
    hit = total = 0
    for xb, yb in loader:
        xb, yb = xb.to(device), yb.to(device)
        hit += (model(xb).argmax(1) == yb).sum().item()
        total += len(yb)
    return hit / total
```

```
# 4. Training.
for epoch in range(20):
    model.train()
    for xb, yb in train:
        xb, yb = xb.to(device), yb.to(device)
        opt.zero_grad()
        loss = loss_fn(model(xb), yb)
        loss.backward()
        opt.step()
    if epoch % 5 == 0:
        print(epoch, round(loss.item(), 3), round(accuracy(val), 3))

print('final accuracy', accuracy(val))
```

B4 A custom module and dataset

```
import torch
from torch import nn
from torch.utils.data import Dataset

class Block(nn.Module):
    def __init__(self, width, p=0.1):
        super().__init__()
        self.net = nn.Sequential(nn.Linear(width, width), nn.ReLU(), nn.Dropout(p))

    def forward(self, x):
        return x + self.net(x)              # a residual connection

class Windows(Dataset):
    """Sliding windows over a series: each item is (past values, next value)."""
    def __init__(self, series, size):
        self.s, self.size = torch.as_tensor(series, dtype=torch.float32), size

    def __len__(self):
        return len(self.s) - self.size

    def __getitem__(self, i):
        return self.s[i:i + self.size], self.s[i + self.size]
```

B5 Overfit one batch

```
xb, yb = next(iter(train))
xb, yb = xb.to(device), yb.to(device)
for step in range(500):
    opt.zero_grad()
    loss = loss_fn(model(xb), yb)
    loss.backward()
    opt.step()
print(loss.item())                    # should be close to zero
```

B6 Inspecting gradients

```
for name, p in model.named_parameters():
    if p.grad is not None:
        print(f'{name:20s} {p.grad.norm().item():.4f}')
```

B7 Mixed precision, clipping and accumulation together

```
use_amp = device == 'cuda'                    # on a CPU the scaler and autocast switch them
scaler = torch.amp.GradScaler('cuda', enabled=use_amp)
accumulate = 4

for i, (xb, yb) in enumerate(train):
    xb, yb = xb.to(device), yb.to(device)
    with torch.amp.autocast('cuda', enabled=use_amp):
        loss = loss_fn(model(xb), yb) / accumulate
    scaler.scale(loss).backward()
    if (i + 1) % accumulate == 0:
        scaler.unscale_(opt)                  # so clipping sees real gradients
        nn.utils.clip_grad_norm_(model.parameters(), 1.0)
        scaler.step(opt)
        scaler.update()
        opt.zero_grad()
```

B8 Checkpoints

```
torch.save({'model': model.state_dict(), 'opt': opt.state_dict(), 'epoch': epoch}, 'ckpt.pt

ckpt = torch.load('ckpt.pt', map_location=device)
model.load_state_dict(ckpt['model'])
opt.load_state_dict(ckpt['opt'])
start = ckpt['epoch'] + 1
```

Part C Practice

P1. What shape results from broadcasting $(5, 1)$ and $(1, 7)$?

$(5, 7)$.

P2. A 10-class classifier starts with a loss of 2.30. Is this a good sign?

Yes. $\ln 10 = 2.30$, which is what an untrained, unbiased model should give.

P3. A model cannot overfit a single batch. What do you conclude?

There is a bug, for example in the data, the loss or the optimiser. A working model can memorise one batch.

P4. Why not apply softmax before `nn.CrossEntropyLoss`?

It applies log-softmax itself, so softmax would be applied twice, flattening the scores.

P5. A loss is computed with shapes (32) and $(32, 1)$. What goes wrong?

Broadcasting produces a 32×32 matrix, so every prediction is compared with every target.

P6. Gradient norm is 10 and the clip limit is 1. By what factor are gradients scaled?

0.1.

P7. You evaluate with dropout still active. What do you see?

Noisy, usually worse, scores, because the model runs with randomly dropped units. Call

`model.eval()`.